

Ex. No. : 06

Date:

Register No.: 231701014

Name: N GOKUL KRISHNA

A Python Program To Do Face Recognition Using SVM Classifier

Aim:

To implement **face recognition using Support Vector Machine (SVM)**.

Requirements:

Jupyter Notebook with Python, fetch_olivetti_faces dataset, and required Python libraries.

PROGRAM:

Step 1: Import Libraries

```
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay
```

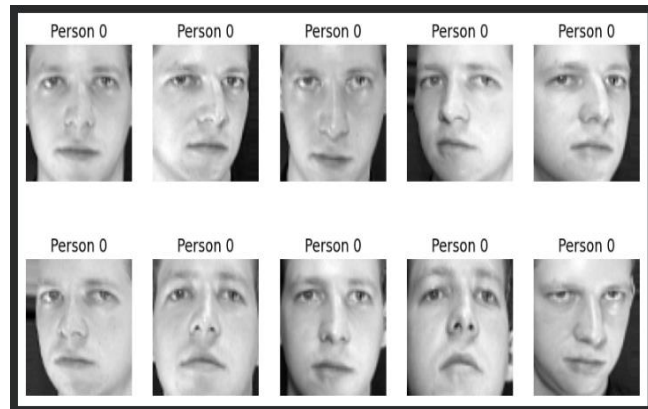
Step 2: Load Dataset

```
faces = fetch_olivetti_faces()
X = faces.data
y = faces.target
target_names = [str(i) for i in set(y)]

print("Dataset shape:", X.shape)
print("Number of classes:", len(set(y)))
```

Step 3: Visualize Sample Faces

```
fig, axes = plt.subplots(2, 5, figsize=(10, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(faces.images[i],
              cmap='gray')
    ax.set_title(target_names[y[i]])
    ax.axis('off')
plt.show()
```



Step 4: Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split( X, y,
                                                    test_size=0.25, random_state=42)
```

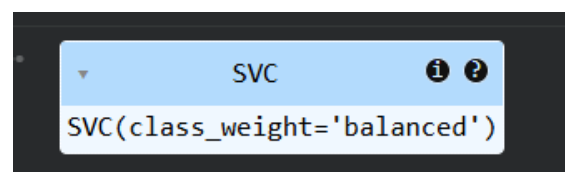
Step 5: Apply PCA (Dimensionality Reduction)

```
pca = PCA(n_components=100, whiten=True, random_state=42)

X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
```

Step 6: Train SVM Classifier

```
model = SVC(kernel='rbf', class_weight='balanced')
model.fit(X_train_pca, y_train)
```



Step 7: Predict Results

```
y_pred = model.predict(X_test_pca)
```

Step 8: Model Evaluation

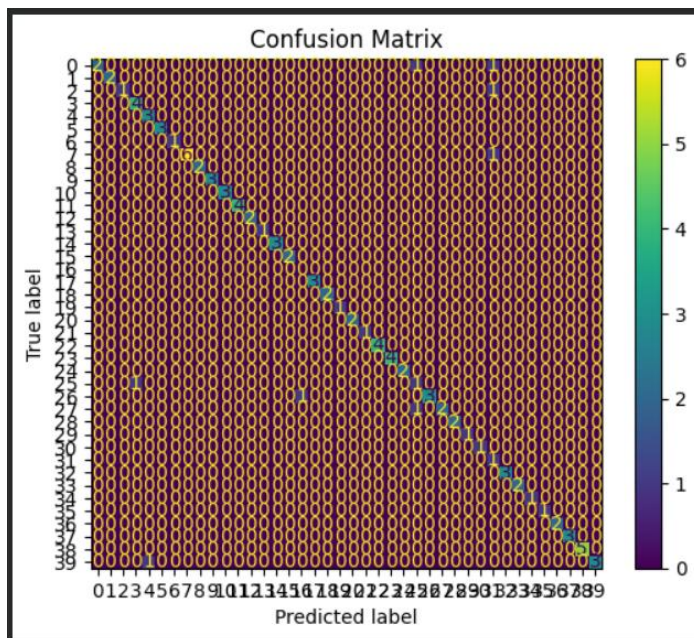
Confusion

MatrixConfusionMatrixDisplay.fro

```
m_predictions(y_test, y_pred)
```

```
plt.title("Confusion Matrix")
```

```
plt.show()
```



Classification Report

```
print(classification_report(y_test, y_pred,  
target_names=target_names))
```

```
print(classification_report(y_test, y_pred))
```

...		precision	recall	f1-score	support
	0	1.00	0.50	0.67	4
	1	1.00	1.00	1.00	2
	2	1.00	0.50	0.67	2
	3	0.80	1.00	0.89	4
	4	0.75	1.00	0.86	3
	5	1.00	1.00	1.00	3
	6	1.00	1.00	1.00	1
	7	1.00	0.86	0.92	7
	8	1.00	1.00	1.00	2
	9	1.00	1.00	1.00	3
	10	1.00	1.00	1.00	3
	11	1.00	1.00	1.00	4
	12	1.00	1.00	1.00	2
	13	1.00	1.00	1.00	1
	14	1.00	1.00	1.00	3

Face Recognition Output:

```
fig, axes = plt.subplots(2, 5, figsize=(10, 5))
```

```
for i, ax in enumerate(axes.flat):
```

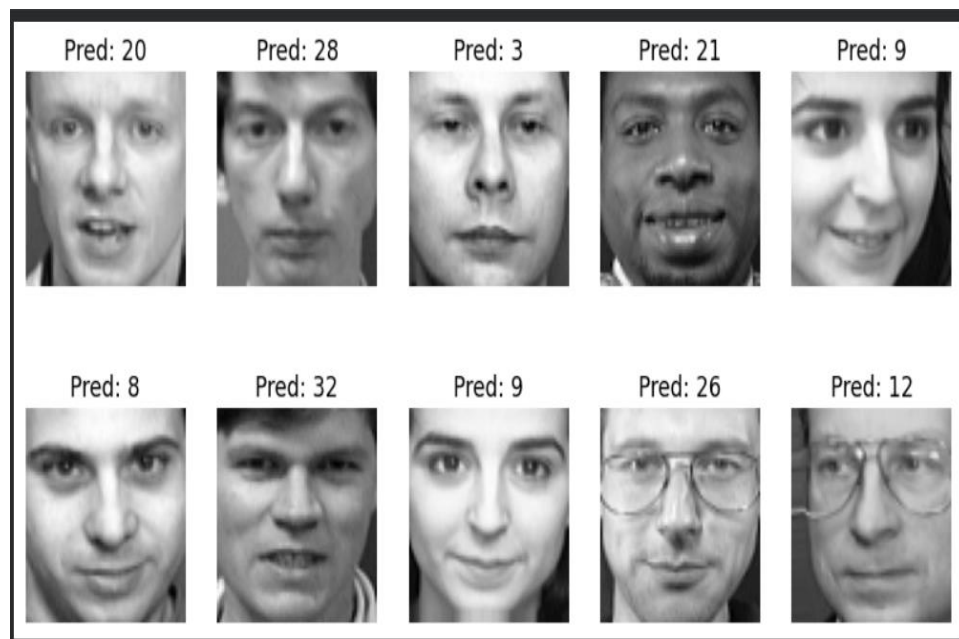
```
    ax.imshow(X_test[i].reshape(faces.images[0].shape), cmap='gray')
```

```
    ax.set_title(f"Pred:
```

```
    {target_names[y_pred[i]]}")
```

```
    ax.axis('off')
```

```
plt.show()
```



Result:

Thus, the Multi Layer Perceptron with Backpropagation was successfully implemented and evaluated using the dataset.